EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 6-IDA: IDA Pro — Read, Hack, and Patch a Static Variable and a GPIO Input (Raw `.bin`)

ARM Cortex-M33 • GDB • Ghidra

October 20, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the lesson project with `Release` and get both an `.elf` and a raw `.bin`
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load the raw `.bin` into IDA at `0x10000000`
- Read how a **static local** keeps a fixed RAM address while a **regular local** is inlined to a constant programs
- **Resolve the functions in the IDA GUI** using the ELF symbol map
- **Patch** the value byte, invert the button/LED logic, and (optionally) rename the printed label
- **Export** the patched image, convert it to UF2, and export it

How This Guide Works

The build produces two files for the project:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table	Ground truth for every function address and name
`.bin`	The raw flash image, no headers, no symbols	The image we load into IDA and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in IDA, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the Week 6 lesson, and the Week 6 lesson is a `Release` build. `Release` optimizes the code the same way the original lesson was built: it **inlines** the `static` helper `demo_static_variable` (and the SDK's `gpio_set_dir` / `gpio_get` / `gpio_put` / `gpio_pull_up` helpers) into `main`, and it folds the regular local `regular_fav_num` down to a bare `movs r1, #42` constant. If you build `Debug`, the SDK function addresses move and the helper stays a separate call, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**:

d prove what the code does. ge. 3. Resolve the functions in IDA using the ELF symbol map. 4. Patch the bytes, export, convert and export.

Project	Prints	Also does	The hacks
`0x0014_static-variables`	`regular_fav_num: 42`, then `static_fav_num:` incrementing	reads button GPIO 15, drives LED GPIO 16	change `42` to `43`, invert the button/LED logic, (optional) rename the label

> **Addresses come from your build.** Every address here is from the `Release` build produced in Step 3. Confirm against your own `.elf` with the command in Step 4.

> `static_fav_num` **is the star of this week.** `regular_fav_num` is an automatic (stack) variable that the compiler bakes into a constant; `static_fav_num` is a function-local `static` that the linker parks at a **fixed RAM address** (`0x200005a8`) in `.data`. That fixed address is why it persists across loop iterations, and why you can read and patch it like a global.

Part 1: Build and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **IDA Pro** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the project with `Release`

Run this inside `0x0014_static-variables/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Point IDA at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so IDA never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the IDA console, so the whole build -> patch -> flash loop stays inside IDA. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"]  # the console's PATH omits Homebrew
proj = os.path.join(root, "0x0014_static-variables")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x0014_static-variables")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x0014_static-variables")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

The build directory now contains the pair we need:

- `0x0014_static-variables/build/0x0014_static-variables.elf` and `.bin` — the `.bin` is **15516** bytes (`0x3c9c`)

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...`:

OS	Typical toolchain path
Windows x64	`C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin`
macOS Apple Silicon	`~/.pico-sdk/toolchain/14_2_Rel1/bin`
Linux x64	`/usr`

> **This guide's toolchain lives at** `~/.pico-sdk/toolchain/14_2_Rel1/bin`. All of `arm-none-eabi-nm`, `arm-none-eabi-objdump`, and `arm-none-eabi-gdb` resolved in this document come from there. If your install is elsewhere, `which arm-none-eabi-nm` tells you where to point.

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on the ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x0014_static-variables.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x0014_static-variables.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T` / `t` type is a function. Here are the functions this lesson uses. The signatures come from the ELF's DWARF debug info queried with `arm-none-eabi-gdb -batch -ex "ptype <name>"`, so they are exact.

Our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (`demo_static_variable` inlined)

The GPIO functions `main` calls (or that are inlined into it):

Address	ELF symbol	Signature	Role
`0x1000029c`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select (UART pins)

Address	ELF symbol	Signature	Role
`0x100002d8`	`gpio_set_pulls`	`void gpio_set_pulls(uint, bool, bool)`	SDK pull config (`gpio_pull_up` inlined)
`0x10000300`	`gpio_init`	`void gpio_init(uint)`	SDK GPIO init

The stdio/UART and printf chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x10000e60`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock
`0x10000ed0`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x10002dbc`	`exit`	`void exit(int)`	C runtime exit
`0x10002dc4`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10002df0`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x10002f00`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output
`0x10002fec`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x10003014`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x100030e0`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`	printf core
`0x100031a4`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper
`0x10003360`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init
`0x100034a0`	`strlen`	`size_t strlen(const char*)`	C runtime string length

SDK helpers the `printf` and `uart_init` paths reach:

Address	ELF symbol	Signature	Role
`0x10002d60`	`vfctprintf`	`int vfctprintf(void (*)(char, void*), void*, const char*, va_list)`	printf format engine
`0x1000237c`	`_vsnprintf`	`int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)`	the format dispatcher
`0x10001760`	`_ntoa_format`	`unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)`	number formatter

Address	ELF symbol	Signature	Role
`0x100016c4`	`_out_rev`	`unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)`	reversed-digit output
`0x10001934`	`_out_char`	`void _out_char(char, void*, size_t, size_t)`	single-char sink
`0x10000e74`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x100010a4`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART clock lookup

Two things in this project have **no symbol of their own**, because the compiler inlined them into `main`:

- `demo_static_variable` — the `static` helper in our own source is inlined, so there is no `demo_static_variable` address to rename. You see its body directly inside `main`.
- `init_gpio`, `gpio_set_dir`, `gpio_get`, `gpio_put`, and `gpio_pull_up` — these are `static inline` in the SDK headers, so they compile to the `mcrr`/SIO writes and the `ldr`/`ubfx` reads you see in `main` rather than to calls. `gpio_pull_up(15)` becomes the direct call to `gpio_set_pulls` at `0x1000024e`.

> `static_fav_num` **is a** `t` **symbol, not a function.** `arm-none-eabi-nm -n` lists `200005a8 t static_fav_num.0`. That lowercase `t` is a **local data** symbol: the linker named the function-local static `static_fav_num.0` and parked it at RAM address `0x200005a8`. There is no pool entry for `regular_fav_num` because it is an automatic (stack) variable — and here the compiler never gave it a stack slot at all, folding it into a constant.

Part 2: Load the Raw `.bin` into IDA

Start from a fresh IDA state. If you already have a database for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into IDA

A raw `.bin` has no headers, so IDA cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, IDA may load it with a guessed architecture, and every address in this lesson will be wrong.

1.1 Open the file

1. **File ▸ Open...** (Ctrl/Cmd+O).
2. Select the `.bin` file and click **Open**.

IDA opens the **"Load a new file"** dialog because a raw `.bin` has no header.

1.2 Set the format and processor

In the *Load a new file* dialog:

1. **Format**: leave it as **Binary file** (IDA auto-detects a headerless image).
2. **Processor type**: click the **...** button next to the processor field. In the *Processor type* dialog:
    - Family: **ARM**
    - Type: **ARM Little-endian** (the 32-bit ARM module).
    - Click **OK**.

> If IDA asks **"Do you want to disassemble it as 64-bit code?"** — click **No**. The RP2350 is 32-bit; answering *Yes* loads AArch64 and you get `X` registers / data. (This is the single most common cause of the `DCB` wall.)

1.3 Configure the ARM architecture and Thumb mode

Still in the *Load a new file* dialog, open the processor options (the **Processor options** / **Set...** button, which opens the dialog titled **ARM architecture options**):

- **ARM architecture**: choose **ARMv8-M** (this is the Cortex-M33 / RP2350 core).
- **Thumb instructions**: select **Thumb-2** (ARMv8-M is Thumb-only, so this is the correct mode).
- Leave **automatic ARM-THUMB switch** off; we want Thumb throughout.

1.4 Set the loading address

- **Loading address**: enter `10000000` (hex; the RP2350 XIP flash base). IDA rejects addresses that don't look like ROM/RAM with *"The loading address should belong to RAM or ROM"* — `0x10000000` is the correct flash window.

Leave **Manual load** and **Create segments** at their defaults. Click **OK**.

1.5 Force Thumb, then seed the code from the vector table

A raw `.bin` has no header, so IDA's loader has **no entry point** and defaults the segment to **ARM mode** (which cannot decode Thumb). Two required steps — in this order:

A. Set the segment to Thumb (do this first).

- **Edit ▸ Segments ▸ Set default segment register value...** → register `T` = `1` (or press `Alt+G`). IDA reanalyzes the segment as Thumb automatically.

B. Seed the code from the vector table.

1. **G** → `10000000`. The first two words are the initial stack pointer and the **reset vector**.
2. **Read the 4-byte word at** `0x10000004` (do **not** assume its value — it differs per build).
3. **Clear bit 0** (the ARM **Thumb** flag) → the reset handler address.
4. **G** to that address → press **C** (**MakeCode**). IDA disassembles the reset handler and follows its `bl`s into `main`.

Once `main` shows code, the import is correct.

Why IDA needs this and others don't: IDA's generic raw-binary ARM loader treats the image as flat data and waits for you to set Thumb and mark the entry point. It's an IDA quirk, not a mistake on your part.

Step 8: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment.

> **macOS function keys:** the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

Part 3: Static — Resolve the Functions in IDA and Patch

Step 16: Resolve the functions in the IDA GUI

We now name the functions in IDA using the ELF symbol map from Step 4. IDA loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	IDA action	Use it for
`G`	Go to address	Jump to a function's address
`Y`	**Change Type**	Set the function's signature. The dialog shows the full prototype, so this sets the name *and* the type in one step.
`N`	Rename	Rename only, when you just want the name and not the type

For each function below: `G` to its address, then `Y` **(Change Type)** and type the prototype from the table.

How to resolve a function in IDA (`Y`)

`Y` is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_10003014()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. `G` to the function's address. The cursor lands on the function.
2. Press `Y`. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

The decompiler header then shows the real prototype, and calls to the function read cleanly instead of `sub_<addr>()`. `N` is only for renaming without touching the type; `Y` alone sets both the name and the type.

If `Y` seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type...**. IDA parses what you type and silently keeps the old type if it does not parse, so glance at the header after each `Y`.

Worked example: `main`

1. Press `G`, type `0x10000234`, press Enter. The view jumps there; the cursor lands on `sub_10000234`.
2. Press `Y` (Change Type), type `int main(void)`, press Enter. That sets the name to `main` and the type to `int(void)`.

> **IDA shows `int32_t` where Ghidra shows `int`.** After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and IDA's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `gpio_init`

1. `G` -> `0x10000300`.
2. `Y` -> `void gpio_init(uint gpio)`.

Worked example: `gpio_set_pulls`

1. `G` -> `0x100002d8`.
2. `Y` -> `void gpio_set_pulls(uint gpio, bool up, bool down)`.

This is what `gpio_pull_up(15)` in our source compiles to: the SDK's `static inline` `gpio_pull_up` disappears, and `main` calls `gpio_set_pulls(15, true, false)` directly at `0x1000024e`.

Worked example: `gpio_set_function`

1. `G` -> `0x1000029c`.
2. `Y` -> `void gpio_set_function(uint gpio, gpio_function_t fn)`.

Worked example: `__wrap_printf`

1. `G` -> `0x100031a4`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic.

> **`printf` in our source is `__wrap_printf` in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper.

The rest of the chain is the same two keystrokes per function (`G`, then `Y`). This is **our code plus the library functions it actually calls** — not the whole SDK. `main` calls `stdio_init_all`, `gpio_init`, `gpio_set_pulls`, and `printf`, so we follow that chain down: `stdio_init_all` pulls in the stdio/UART setup, `printf` lands in the SDK's `__wrap_printf`, and `uart_init` reaches `clock_get_hz` and `busy_wait_us`.

The call chain for this project:

```
main
├── stdio_init_all
│   └── stdio_uart_init ── gpio_set_function, uart_init, stdio_set_driver_enabled
│                              └── uart_init ── clock_get_hz, busy_wait_us
├── gpio_init
├── gpio_set_pulls              (gpio_pull_up inlined)
├── __wrap_printf ── __wrap_vprintf
│   ├── vfctprintf ── _vsnprintf
│   │   └── _ntoa_format / _out_rev / _out_char
│   ├── stdio_out_chars_crlf
│   └── time_us_64
└── gpio_set_dir / gpio_get / gpio_put   (inlined; no calls)
```

Resolve every function in that chain:

Address	Rename to (N)	Signature (Y)
0x1000015c	_reset_handler	void _reset_handler(void)
0x10000186	platform_entry	void platform_entry(void)
0x1000019a	data_cpy	void data_cpy(void*, void*, void*)
0x100001e4	_init	void _init(void)
0x10000210	frame_dummy	void frame_dummy(void)
0x10000234	main	int main(void)
0x1000029c	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x100002d8	gpio_set_pulls	void gpio_set_pulls(uint, bool, bool)
0x10000300	gpio_init	void gpio_init(uint)
0x10000e60	time_us_64	uint64_t time_us_64(void)
0x10000ed0	uart_init	uint uart_init(uart_inst_t*, uint)
0x10002dbc	exit	void exit(int)
0x10002dc4	runtime_init	void runtime_init(void)
0x10002df0	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10002f00	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x10002fec	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x10003014	stdio_init_all	bool stdio_init_all(void)
0x100030e0	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x100031a4	__wrap_printf	int __wrap_printf(const char*, ...)
0x10003360	stdio_uart_init	void stdio_uart_init(void)
0x100034a0	strlen	size_t strlen(const char*)

Resolve the SDK helpers the `printf` and `uart_init` paths reach:

Address	Rename to (N)	Signature (Y)
0x10002d60	vfctprintf	int vfctprintf(void (*)(char, void*), void*, const char*, va_list)
0x1000237c	_vsnprintf	int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)
0x10001760	_ntoa_format	unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)
0x100016c4	_out_rev	unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)

Address	Rename to (N)	Signature (Y)
`0x10001934`	`_out_char`	`void _out_char(char, void*, size_t, size_t)`
`0x10000e74`	`busy_wait_us`	`void busy_wait_us(uint64_t)`
`0x100010a4`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`

A `void` return type may not stick — here is the fix. IDA treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type IDA thinks it can infer.

Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:

```
from binaryninja import ReturnValue, Type
fn = bv.get_function_at(0x1000015c)
if fn is not None:
    fn.return_value = ReturnValue(Type.void())
```

That holds `_reset_handler` at `void` even after reanalysis. If it still will not stick, leave it — it does not affect the rest of the lesson.

`__wrap_printf` is the real symbol. `printf` in our source compiles to the SDK's `__wrap_printf` (which forwards to `__wrap_vprintf`). Rename it `printf` if you prefer the lesson's shorthand, but `__wrap_printf` is what the ELF says.

`stdio_init_all` returns `bool`, not `void` — `_Bool stdio_init_all(void)` in the ELF.

Shortcut — resolves name *and* type for every function. Instead of doing `N` + `Y` by hand, paste this into IDA's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
typedef void (*out_fct_type)(char, void*, size_t, size_t);
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x1000029c: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x100002d8: ("gpio_set_pulls",            "void gpio_set_pulls(uint, bool, bool)"),
    0x10000300: ("gpio_init",                 "void gpio_init(uint)"),
    0x10000e60: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x10000ed0: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x10002dbc: ("exit",                      "void exit(int)"),
    0x10002dc4: ("runtime_init",              "void runtime_init(void)"),
    0x10002df0: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10002f00: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x10002fec: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10003014: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x100030e0: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x100031a4: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x10003360: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x100034a0: ("strlen",                    "size_t strlen(const char*)"),
    0x10002d60: ("vfctprintf",                "int vfctprintf(void (*)(char, void*), void*, const
char*, va_list)"),
    0x1000237c: ("_vsnprintf",                "int _vsnprintf(out_fct_type, char*, size_t, const
char*, va_list)"),
    0x10001760: ("_ntoa_format",              "unsigned _ntoa_format(out_fct_type, char*, size_t,
size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)"),
    0x100016c4: ("_out_rev",                  "unsigned _out_rev(out_fct_type, char*, size_t,
size_t, const char*, size_t, unsigned, unsigned)"),
    0x10001934: ("_out_char",                 "void _out_char(char, void*, size_t, size_t)"),
    0x10000e74: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x100010a4: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
    if f is not None:
        f.set_user_type(sig)
```

> SDK type names (`stdio_driver_t` , `gpio_function_t` , `uart_inst_t` , plus `uint` , `va_list` , `out_fct_type` , and `clock_handle_t`) are **not** in the raw `.bin` . `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first (an opaque `struct` / `enum` / `typedef` is enough to parse). If you add a function that uses another SDK type, add a definition for it to that block too.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main` . Once the functions above are typed, it reads roughly:

```
int32_t main(void)
{
    stdio_init_all();
    gpio_init(0xf);
    gpio_set_dir(0xf, GPIO_IN);          // inlined: mcrr 0, 4, r0, r3, cr4
    gpio_set_pulls(0xf, true, false);    // gpio_pull_up(15)
    gpio_init(0x10);
    gpio_set_dir(0x10, GPIO_OUT);        // inlined
    do
    {
        __wrap_printf("regular_fav_num: %d\r\n", 0x2a);
        __wrap_printf("static_fav_num: %d\r\n", *(uint8_t*)0x200005a8);
        *(uint8_t*)0x200005a8 = *(uint8_t*)0x200005a8 + 1;   // static_fav_num++
        gpio_put(0x10, gpio_get(0xf) ^ 1);                    // inlined SIO read/write
    } while (true);
}
```

The `0x2a` is the value we edited live; `0x200005a8` is the fixed RAM address of `static_fav_num` . The `gpio_get(0xf) ^ 1` is the `eor.w r3, r3, #1` we will patch in Step 18b. Now make the hacks permanent.

Step 18: Patch 1 — change `regular_fav_num` from 42 to 43

Go to `0x10000264` :

```
10000264: 212a   movs  r1, #42 ; 0x2a
```

The halfword is `0x212a` , stored little-endian as `2a 21` . The immediate is the low byte, so the byte at the instruction's own address is `0x2a` . Change it to `0x2b` (43).

Project	Address	Before	After	Effect
`0x0014`	`0x10000264`	`2a`	`2b`	`movs r1, #42` -> `#43` , prints `regular_fav_num: 43`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Toggle the lock off so editing is enabled.
3. Go to `0x10000264` and change the byte `2A` to `2B` .
4. Return to the linear view, right-click the function -> `Reanalyze` .

Option B — Python console:

```
bv.write(0x10000264, b"\x2b")
print(hex(bv.read(0x10000264, 1)[0]))  # -> 0x2b
```

After reanalysis the instruction reads `movs r1, #43`.

Step 18b: Patch 2 — invert the button/LED logic

The stock code inverts the raw button bit with `eor.w r3, r3, #1` at `0x10000286`. Its four bytes are `83 f0 01 03`; the immediate `#1` is the **third** byte, at `0x10000288`. Change `01` to `00` so the XOR becomes `#0` (a no-op) and the LED follows the raw pin state instead:

```
10000286: f083 0301   eor.w  r3, r3, #1 ; immediate byte at 0x10000288
```

Project	Address	Before	After	Effect
`0x0014`	`0x10000288`	`01`	`00`	`eor.w r3, r3, #1` -> `#0`, inverts the LED behavior

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10000288` and change the byte `01` to `00`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x10000288, b"\x00")
print(bv.read(0x10000286, 4).hex())  # -> 83f00003
```

Now the logic is permanently changed:

- Button released (GPIO 15 reads `1`): `1 XOR 0 = 1` -> LED **ON**
- Button pressed (GPIO 15 reads `0`): `0 XOR 0 = 0` -> LED **OFF**

This is the **opposite** of the original behavior. The `gpio_get` and `gpio_put` are inlined, so the only byte that controls the inversion is this immediate.

Step 18c: Patch 3 — rename the printed label (optional)

The format string `"regular_fav_num: %d\r\n"` starts at `0x10003560`. Its first 16 bytes are `72 65 67 75 6c 61 72 5f 66 61 76 5f 6e 75 6d 3a` (`regular_fav_num:`). Change them to `70 61 74 63 68 65 64 5f 66 61 76 5f 6e 75 6d 3a` (`patched_fav_num:`), leaving the `%d\r\n` tail untouched, so the line prints `patched_fav_num: 43`.

Project	Address	Before	After	Effect
`0x0014`	`0x10003560`	`72 65 67 75 6c 61 72 5f 66 61 76 5f 6e 75 6d 3a`	`70 61 74 63 68 65 64 5f 66 61 76 5f 6e 75 6d 3a`	`regular_fav_num:` -> `patched_fav_num:`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10003560` and change the sixteen bytes `72 65 67 75 6c 61 72 5f 66 61 76 5f 6e 75 6d 3a` to `70 61 74 63 68 65 64 5f 66 61 76 5f 6e 75 6d 3a`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x10003560, b"patched_fav_num:")
print(bv.read(0x10003560, 22))  # -> b'patched_fav_num: %d\r\n\x00'
```

Keep the replacement exactly sixteen bytes — the same length as `regular_fav_num:`. If you use a shorter string you must pad it, or `%d` shifts and `printf` reads the wrong argument. A longer string would overwrite the `%d` tail.

Step 19: Export the patched `.bin`

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x0014_static-variables", "build"), "0x0014_static-
variables-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 15516 /.../build/0x0014_static-variables-h.bin
```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base IDA loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x3c9c` = 15516). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.
- Reading `seg.start` for `seg.data_length` bytes therefore grabs exactly the image.

Two gotchas this avoids:

- **No relative path.** IDA's Python console runs with a read-only working directory (inside the app bundle), so `open("0x0014_static-variables-h.bin", "wb")` fails with `OSError: [Errno 30] Read-only file system`. `root` (from `~/.embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/` — no machine-specific path and no database needed.
- **Read the image, not the whole view.** `bv.read(bv.start, bv.length)` spans the entire mapped range, which is not the image. The segment's `data_length` is the image size.

A different size means you exported a partial view.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0014_static-variables-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x0014_static-variables-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

> **Or convert from the IDA console** — it is a normal Python interpreter, so you never have to leave the app. `chdir` to a writable directory first (the default one is read-only), then run the script:
>
> ```
> import os, sys, runpy
> os.chdir(os.path.join(root, "0x0014_static-variables", "build"))   # the project build dir
> (writable)
> sys.argv = ["uf2conv.py", "0x0014_static-variables-h.bin",
>             "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
> runpy.run_path("../../uf2conv.py", run_name="__main__")   # path to your uf2conv.py
> ```
>
> This writes `hacked.uf2` next to the `.bin`, ready to drag onto the Pico.

Static stopping point

The patched image has been exported and converted to UF2. This IDA guide ends at static verification; it does not connect to, flash, or debug a live target.

Glossary

Term	Definition
`.bss`	Section for uninitialized (or zero-initialized) static/global variables; zeroed by startup code
`.data`	Section for initialized static/global variables; copied from flash to SRAM at boot
`.elf`	Linked image with the symbol table; the ground truth for addresses and names
`.rodata`	Read-only section for constants and string literals; stays in flash
Automatic variable	A local variable created and destroyed with its block; lives on the stack (or is optimized away)
`eor` **/ XOR**	Exclusive OR — flips bits where the operands differ
GPIO	General Purpose Input/Output — controllable pins on the microcontroller
ly flash	
Inlining	The optimizer replacing a function call with the function body; why `demo_static_variable` disappears
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr`
MMIO	Memory-mapped I/O — hardware registers accessed as memory addresses
Pull-up / pull-down	A resistor that holds an input pin at a defined level when nothing drives it
SIO	Single-cycle I/O — the fast GPIO block in the RP2350, at `0xd0000000`
Static variable	A variable with static storage duration; persists for the whole program and keeps a fixed address
Ternary operator	`condition ? value_if_true : value_if_false`

Term	Definition
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
`ubfx`	Unsigned Bit Field Extract — pulls a bit field out of a register
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Verify the static behavior from the recovered instructions.